Table of Contents

Declaration i

Abstract ii

List of Tables and Figures iii

List of Symbols and Abbreviations iv

Chapter 1: Introduction

1.1 Project Introduction 1

1.2 Problem Statement 1

1.3 Motivation 2

1.4 Objectives 2

1.5 Scope and Limitations 3

1.6 Contributions of the Project 3

Chapter 2: Literature Survey

2.1 Phishing and URL-Based Attacks 4

2.2 Existing Detection Approaches 4

2.3 Machine Learning for URL Classification 5

2.4 Research Gap 6

2.5 Proposed Approach 6

2.6 Summary of the Survey 7

Chapter 3: Requirements and Methodology

3.1 Functional Requirements 8

3.2 Non-Functional Requirements 8

3.3 Hardware and Software Requirements 8

3.4 Dataset Description 9

3.5 Feature Engineering 9

3.6 Machine Learning Algorithms Considered 11

3.7 Evaluation Methodology 11

3.8 Threshold and Risk Score Interpretation 12

Chapter 4: System Design

4.1 System Architecture 13

4.2 Data Flow Diagram 13

4.3 Sequence of Operations 13

4.4 Database Design 14

4.5 API Design 14

4.6 Security Design 14

Chapter 5: Implementation Plan

5.1 Project Structure 16
5.2 Feature Extraction Module 16
5.3 Model Training Pipeline 16
5.4 Prediction Service 17
5.5 Frontend Dashboard 17

Chapter 6: Testing Plan

6.1 Testing Strategy 18
6.2 Test Case Summary 18
6.3 Security and Performance Testing 18

Chapter 7: Results

7.1 Phishing URL Classification 20
7.2 Machine Learning Model Evaluation 20
7.3 Feature and Threshold Analysis 20
7.4 Application and Dashboard 20
7.5 Security, Testing and Performance 21

Chapter 8: Conclusion and Future Enhancement

8.1 Conclusion 22
8.2 Future Work 22

References 24

Annexure

A. Complete URL Feature Schema 39
B. Development Schedule 40

Abstract

A phishing website deceives users of its authenticity and lures them into disclosing their passwords, card numbers, and other confidential information using a copy of an authentic website. The majority of detection systems rely on third-party services such as a blocklist or live page analysis; both require connectivity to the web and are not able to keep up with newly registered phishing domains. A blocklist can protect users from the URL only after someone has reported and verified it. In that case, there will be a window of vulnerability when a phishing page is still new.

In this project, the goal is to develop a phishing URL detector, which will use only the text of a URL as an input data. The system will extract around 30 features from the text of a URL in terms of length, composition, subdomains, and entropy and pass them through a machine-learning algorithm trained on publicly available collections of phishing and authentic URLs. The algorithm will have no need to fetch or visit a webpage in question and will work offline after training regardless of a response time of the target server.

The following five algorithms will be analyzed: Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, and XGBoost. The evaluation will involve stratified cross-validation, feature selection, and feature ablation studies, when groups of features are gradually removed in order to reveal the most important group for classification. The system will predict the class of a URL and return the probability score and a short explanation based on features through a FastAPI backend and a Next.js frontend dashboard that will also store the history of predictions locally.

All metrics reported in the final project report will be derived from experiments performed with the help of the dataset used in this project. No accuracy, precision or recall estimates will be assumed at this point.

List of Tables

List of Tables

Table 2.1: Comparison of Detection Approaches 5
Table 3.1: Dataset Summary 9
Table 3.2: Complete URL Feature Schema 9
Table 3.3: Algorithms and Tuned Hyperparameters 11
Table 3.4: Evaluation Metrics and Definitions 12
Table 6.1: Sample Test Cases 18
Table Annexure-B: Ten-Week Development Schedule 24

List of Symbols and Abbreviations

MCA : Master of Computer Applications

URL : Uniform Resource Locator

ML : Machine Learning

API : Application Programming Interface

REST : Representational State Transfer

SVM : Support Vector Machine

ORM : Object-Relational Mapping

ROC-AUC : Receiver Operating Characteristic, Area Under Curve

PR-AUC : Precision-Recall, Area Under Curve

TP / TN / FP / FN : True Positive / True Negative / False Positive / False Negative

CV : Cross-Validation

HTTP : Hypertext Transfer Protocol

JSON : JavaScript Object Notation

Chapter 1: Introduction

1.1 Project Introduction

Phishing is a type of social engineering where the attacker creates a fake copy of a website that the user trusts and attempts to lure them into providing login credentials, payment card numbers, or some other kind of sensitive information. Phishing continues to be one of the leading ways of breaking into networks and systems, because it attacks the user directly, rather than some vulnerability in the system. The best way to detect whether a page is fake is not in its appearance, but in how its address was constructed.

This project is called "Phishing Website Detection Using Machine Learning Based on URL Lexical Analysis". It creates a classification of a URL as legitimate or phishing based only on the textual analysis of the URL itself. No page retrieval, browser execution or access to a third-party reputation service are required. This allows the model to stay performant and works without an internet connection, as well as being independent from any external services, even those which may not classify a newly created phishing website yet. An inference is performed in milliseconds, because the whole process works solely with the URL string, and no communication happens outside of the local machine.

The project is an example of cybersecurity application of machine learning. Feature engineering, rigorous experiment design and proper secure coding practices will all be needed to create the system. All of them will be explained throughout the following chapters.

1.2 Problem Statement

The current detection methods of phishing mainly base their analysis either on the use of a blocklist, or on the live analysis of the web page's content. The former method cannot cope with the latest phishing websites that become registered after the last update of the blocklist, since the URL needs to be marked as malicious first and then included in the list. In case of the live analysis, it is necessary to open the page which poses a certain threat to the user, since it might include some exploit, and it requires network access and availability of the targeted server.

Therefore, there is a demand for developing a solution which allows detecting whether a URL belongs to a phishing website based only on the information contained in the string of the URL

itself and without the use of an additional, updated database of known phishing domains. It must be able to analyze the URL offline and quickly enough to use it before clicking the link.

1.3 Motivation

In URLs of malicious websites, one can notice some structural signals which distinguish phishing pages from legitimate ones, e.g. unusually long URL, too many subdomains, encoding which aims at concealing the actual destination, the use of a raw IP address instead of a domain name, and the abundance of digits compared to letters. While each signal separately cannot be seen as proof of phishing nature of the website, they together form a distinct pattern recognizable by the machine-learning model.

It is especially appropriate in the case when many signals need to be combined into one classification criterion and possibly some interactions between the signals have to be accounted for. Developing the system is going to provide practical experience in feature engineering, model selection, evaluation approach in class imbalance setting, and secure system design, all of which are vital in security-related applied machine learning projects.

1.4 Objectives

1. Develop and apply a feature extractor which computes about 30 features from a URL string lexically and structurally without visiting the URL on the network.
2. Construct a dataset of phishing and non-phishing URLs, cleaning, removing duplicates, canonicalizing, and splitting into a training and testing dataset.
3. Train and compare five different machine learning algorithms: Logistic Regression, Decision Tree, Random Forest, SVM, and XGBoost.
4. Perform model evaluation using cross validation and test set, also applying metrics that fit the problem of security classification where a false negative is highly expensive.
5. Conduct a feature ablation test to understand the impact of different groups of features on classification accuracy.
6. Deploy the predictive model behind a FastAPI interface and visualize its output in a Next.js dashboard, providing a probability score and short explanation based on features.
7. Document the whole process in order for the experiment to be reproducible from the same data and random seed.

1.5 Scope and Limitations

The framework includes the processes of URL input, URL validation, feature extraction, machine learning model training, predictions and a web based user interface, and is self-contained, with no need to access any external services or APIs. Live webpage crawlers, HTML and JavaScript parsing and analysis, DNS and WHOIS information retrieval, screen shot

classification or any browser-based blocks are not included, as they were mentioned in Chapter 7 as future work, due to the fact that implementing them would take the project out of URL-only scope set for this synopsis.

As the machine learning algorithm relies only on the URL string, it will not be able to recognize a phishing page being located on an infected legitimate domain name, because the URL alone will provide no information about that infection. Furthermore, it will not be possible to confirm the maliciousness of a given URL with a certainty, because its output is a probability.

1.6 Contributions of the Project

This work combines three innovations that fit into the realm of an MCA final year project submission. Firstly, a schema of a feature that uses URLs only and does not suffer from the problem of training/inference mismatch typical for datasets with features extracted both from URLs and web pages. Secondly, an experimental setup comparing five algorithms under one methodology which includes cross-validation, ablation test and thresholds analysis such that the final choice of algorithm will be based on empirical evidence and not on a conjecture stated in other works. And thirdly, an example of end-to-end solution which includes feature extraction, API serving and a dashboard.

Chapter 2: Literature Survey

2.1 Phishing and URL-Based Attacks

A phishing attempt usually involves tricking an individual into going to a bogus site via a misleading link that may be sent through email, SMS, or from an infected advertisement. Attackers mask their destination by employing certain repeated tricks. The following are a few examples of such tricks. Learning these tricks by name gives context as to why it makes sense to use URL features to perform classification.

Technique	Description	Example Pattern
Typosquatting	A domain that misspells a known brand, such as an extra or swapped letter	amaz0n-login.com
Subdomain stuffing	The real brand name is placed in a subdomain of an unrelated domain	paypal.com.verify-account.net
IP-based hosting	A raw IP address replaces the domain name entirely	http://192.168.1.1/login
URL shortening	A shortening service hides the true destination behind a short link	bit.ly/3xample
Homograph substitution	Visually similar Unicode characters replace Latin letters	paypal.com (Cyrillic a)
Excessive encoding	Percent-encoding is used to obscure characters from casual inspection	http://ex%61mple.com/login

Table 2.1 shows common URL manipulation techniques used in phishing campaigns.

Both typosquatting and subdomain stuffing work on the premise that the average Internet user scans through only the first few characters of a domain name and accepts it as being legitimate as long as there is a recognizable brand name anywhere in the URL. On the other hand, IP address based hosting and percent-encoding have much more to do with escaping pattern matching filters rather than fooling the human eye. These approaches alter specific characteristics of a URL like length or the number of subdomains used.

2.2 Existing Detection Approaches

Four broad families of detection approach appear in current practice and in prior academic work: blocklist-based checking, content-based analysis, DNS or WHOIS-based reputation checking, and URL lexical analysis. Table 2.2 summarizes the trade-offs between them.

Approach	How It Works	Strength	Weakness
Blocklist-based	Checks the URL against a maintained database of known phishing sites	High accuracy for known sites	Blind to new or recently registered domains
Content-based	Loads the page and inspects HTML, JavaScript, or a screenshot	Detects sites regardless of URL structure	Requires opening the page; slower; carries exposure risk

DNS or WHOIS-based	Checks domain age, registrar, and hosting reputation	Effective against very new domains	Requires external lookups; some data is rate-limited or paid
URL lexical analysis	Classifies the URL text using structural and character-level features	Works before the page is opened; offline-capable	Cannot see page content; limited against compromised legitimate domains

Table 2.2 compares detection approaches by mechanism, strength, and weakness.

Blocklist-based approaches, including Google Safe Browsing, compare the URL with a blocklist that contains URLs of sites that have been found to be phishing pages. Such an approach is reliable if the URL has been found in the list, but unreliable if the URL is registered recently, since the blocklist needs to be updated through a process of detection and confirmation. The content-based approach studies the HTML, the JavaScript code, and even a screenshot of the page, and hence is reliable, although it is slower since it needs to fetch the page.

The DNS and WHOIS-based approaches examine factors such as the age of the domain registration, the reputation of the registrar, and the host service provider on the assumption that the phishing domains tend to be registered just before they begin to function. The factors used in such approaches are quite reliable but tend to require an external query and sometimes WHOIS information is not available due to recent policy changes by registrars. The URL lexical approach used in this project stands out since it needs nothing but the URL itself.

2.3 Machine Learning for URL Classification

A number of classical machine learning techniques have been considered for the task of URL-based phishing detection in previous research, including Logistic Regression, Decision Trees, Random Forests, and Support Vector Machines. Logistic Regression has been used extensively as a benchmark due to the speed of training and interpretability of its coefficients, which however cannot model non-linear interactions between predictors and the response if the predictors are not pre-transformed.

Decision Trees and Random Forests can model non-linear boundaries without pre-transforming predictors, and Random Forests have in particular been noted to perform well on structured, tabular data from feature sets such as the ones created through URL lexicon analysis, due to the reduced variance from averaging many trees. Support Vector Machines with the RBF kernel can learn non-linear boundaries, but are more dependent on scaling and tuning parameters C and gamma, as well as scaling up to very large training sets.

The gradient boosting algorithms, of which XGBoost is one of the most popular, construct trees one by one, where each subsequent tree fixes the mistakes made in the previously constructed set of trees. This technique usually produces good performance for the kinds of tabular data available in the current dataset; however, it requires more hyperparameter tuning and is more likely to overfit if tree size and number of trees are not regulated using cross-validation. Another research direction that emerged lately includes deep learning algorithms such as character-level convolutional network and URLNet, which extract features directly from the unprocessed URL string instead of the traditional approach, where engineered features are used. Such algorithms allow capturing character-level information but require more data and computing resources compared to the traditional classifiers used in this project, and this is why they are mentioned as future research directions.

2.4 Research Gap

Several existing feature sets include features based solely on URL data together with features that need the web page to be loaded, such as the number of external URLs, presence of a login form on the HTML page, or the number of embedded objects on the web page. The model trained using such a feature set cannot be applied for prediction on an unopened URL because the features extracted from the web page will automatically drop out of consideration, and there will be a difference between the feature distributions at training time and inference time. Such a problem is not explicitly mentioned in any of the surveyed works despite the fact that it directly affects the validity of the accuracy reported.

In this work, the restriction is imposed on all of the features in order to make sure that each of them uses only URL data and also the same feature extraction function is used in the training and prediction pipelines.

2.5 Proposed Approach

The system that has been proposed extracts a pre-defined set of URL-only lexical and structural features, evaluates the performance of five machine learning models using the same evaluation process that uses cross-validation and an unaltered test set, and provides measurements only after completing the experimentation phase. Feature selection and ablation are done to justify the impact of feature sets in the classification process and not rely on a black box approach. This way, the project remains academic in nature as all claims about the performance of the model have been validated through measurement and not from previous research.

2.6 Summary of the Survey

From the literature review, there are three recommendations that could be drawn for this project; use only URLs as features so as to eliminate the train-inference mismatch discussed in Section 2.4, try out more algorithms besides Random Forest or XGBoost in the hope that one of them would beat both of them and consider the false negative rate separately from the accuracy rate, owing to its higher materiality.

Chapter 3: Requirements and Methodology

3.1 Functional Requirements

- The system will take in an URL as an input and verify its length and format.
- The system will extract a predefined set of lexical and structural features from the input URL through one deterministic function.
- The system will predict whether the input URL is legitimate or phishing and give a probability score along with the output class.
- The system will show indicators of feature values based on which the prediction was made, but not with absolute confidence.
- The system will save the prediction history locally, indexed by the URL in hashed form.
- The system will provide model information, such as the algorithm used, training date, and evaluation metrics in a separate page.

3.2 Non-Functional Requirements

- The system should refrain from making any network calls to the provided URL at prediction time in all scenarios.
- The system shall be completely hosted on local infrastructure and should not depend on any commercial API service.
- The system should deliver the result of prediction for a URL in a reasonable time latency for an interactive experience.
- The feature extraction code used in training the system should also be used for prediction, without duplication of code.
- The system shall maintain a record of all administrative operations in an audit log.

3.3 Hardware and Software Requirements

Hardware: A typical development workstation with at least 8 GB of memory is enough to run the training process on datasets mentioned in Section 3.4. GPUs are not needed for any of the five algorithms covered by this project, because none of them is a deep neural network.

Software: Python 3.x, scikit-learn, pandas, NumPy, and joblib are used for the machine learning part, while XGBoost will be additionally required for the gradient boosting algorithm. The backend is served by FastAPI and Uvicorn, request/response schemas are validated by Pydantic. The Next.js, React, and TypeScript libraries create the frontend dashboard and Tailwind CSS is used to style it. Database: PostgreSQL or SQLite; Testing: pytest and FastAPI TestClient; Version control: Git.

3.4 Dataset Description

This project will utilize public phishing URL data sets, for instance, the UCI Phishing Websites Data Set that comprises of 11,055 cases with 30 documented attributes and PhiUSIIL Phishing URL Data Set with larger number of cases and other attributes. Table 3.1 highlights the documentation process of the dataset that will be utilized.

Item	Planned Documentation
Source	UCI Phishing Websites Dataset and/or PhiUSIIL Phishing URL Dataset
License	Recorded from the dataset's published source page
Sample count	Recorded after cleaning and deduplication
Class distribution	Recorded as phishing vs. legitimate counts
Label definition	As defined by the original dataset publisher
Duplicate policy	Exact duplicate URLs removed; conflicting labels flagged
Train/test split	80% training, 20% held-out test, stratified by label

Table 3.1 lists the dataset documentation items to be completed once the dataset is finalized.

Anything that is present as a feature in the source dataset but cannot be derived from just the URL string, like any attribute that can be obtained from a webpage such as the number of external links, will be excluded prior to training. This ensures that the features in both training and testing are identical, which is precisely the problem that we have recognized in Section 2.4 of our literature review.

Preprocessing of the source data will be done in a consistent sequence: removing any entries with invalid or missing URLs, removing any exact duplicate URLs, normalizing the label column to two consistent classes, canonicalizing the URL strings so that different trivial versions of the same URL are not considered distinct entries, and resolving any conflicts between labels for the same canonical URL. Only then will the stratified train/test split be performed, and the feature scaling done on the training portion alone, not the full dataset, to avoid test set leakage into the training process.

3.5 Feature Engineering

About 30 URL-only characteristics will be calculated per URL, divided into the following six groups: length characteristics, character count characteristics, host structure, query structure, entropy characteristics, and suspicious keyword characteristics. The same six groups are also employed later on in the ablation test in the full report of the project. Table 3.2 shows the full schema.

Feature	Description	Type	Rationale
url_length	Total character count of the URL	Integer	Phishing URLs are often longer, to hide the true domain
hostname_length	Character count of the hostname portion	Integer	Long hostnames can indicate stuffed subdomains

path_length	Character count of the path portion	Integer	Unusually long paths can encode extra tracking or obfuscation
query_length	Character count of the query string	Integer	Long queries can carry obfuscated redirect targets
num_digits	Count of digit characters in the URL	Integer	Phishing URLs often substitute digits for letters
num_letters	Count of alphabetic characters	Integer	Baseline for character-ratio features
num_special_chars	Count of non-alphanumeric characters	Integer	High counts can indicate obfuscation
num_dots	Count of '.' characters	Integer	Correlates with subdomain depth
num_hyphens	Count of '-' characters	Integer	Common in typosquatted brand names
num_slashes	Count of '/' characters	Integer	Correlates with path depth
num_subdomains	Number of subdomain levels before the registered domain	Integer	Excessive subdomains can bury the real host
has_ip_host	Whether the host is a raw IP address	Boolean	Legitimate sites rarely use a bare IP as the visible host
has_at_symbol	Whether '@' appears in the URL	Boolean	Text before '@' is ignored by browsers, a known obfuscation trick
has_port	Whether an explicit port number is present	Boolean	Nonstandard ports are uncommon on legitimate consumer sites
has_userinfo	Whether userinfo (user:pass@) is present	Boolean	Rare in legitimate URLs; used to mislead the reader
has_fragment	Whether a '#' fragment is present	Boolean	Occasionally used to append junk to a copied URL
has_query	Whether a query string is present	Boolean	Baseline flag for query-related features
num_query_params	Count of query parameters	Integer	Many parameters can indicate a redirect chain
num_encoded_chars	Count of percent-encoded characters	Integer	Heavy encoding can obscure the true text of the URL
has_double_slash_path	Whether '//' appears inside the path	Boolean	Sometimes used to smuggle a second embedded URL
digit_ratio	Ratio of digits to total URL length	Float	A high ratio is atypical for legitimate brand domains
special_char_ratio	Ratio of special characters to total length	Float	A high ratio suggests obfuscation
path_to_url_ratio	Ratio of path length to total URL length	Float	Captures how much of the URL is consumed by the path
longest_host_token	Length of the longest dot-separated hostname token	Integer	Long single tokens can be randomly generated subdomains
longest_path_segment	Length of the longest slash-separated path segment	Integer	Long segments can carry encoded payloads
hostname_entropy	Shannon entropy of the hostname string	Float	Randomly generated hostnames have higher entropy
url_entropy	Shannon entropy of the full URL string	Float	General measure of character unpredictability
suspicious_keyword_count	Count of terms such as 'login', 'verify', 'secure'	Integer	These terms recur in credential-harvesting pages

has_punycode	Whether the hostname uses punycode (xn--) encoding	Boolean	A known vector for homograph attacks
has_hex_token	Whether a hexadecimal-looking token appears in the URL	Boolean	Can indicate an auto-generated or encoded path segment

Table 3.2 lists the complete URL-only feature schema planned for this project.

All elements of Table 3.2 are generated based on the input URL string by utilizing Python's default urllib.parse library, without resolving DNS and without making an HTTP request. There will be a limit of 4096 characters for the length of input to ensure there won't be any error during parsing, and invalid input will generate a validation error instead of program crash. None of the features listed in this table is considered a single proof of phishing.

3.6 Machine Learning Algorithms Considered

Five algorithms will be trained and compared under the same cross-validation protocol. Table 3.3 lists the hyperparameters planned for tuning and any required preprocessing step for each.

Algorithm	Hyperparameters to Tune	Preprocessing
Logistic Regression	C (inverse regularization strength)	StandardScaler applied before fitting
Decision Tree	max_depth, min_samples_leaf	No scaling required
Random Forest	n_estimators, max_depth, min_samples_leaf, max_features	No scaling required
SVM (RBF kernel)	C, gamma	StandardScaler applied before fitting
XGBoost	n_estimators, max_depth, learning_rate, subsample	No scaling required

Table 3.3 summarizes the tuning plan for each candidate algorithm.

The final model is to be chosen based on cross-validation and testing sets rather than from assumptions made in advance based on the literature covered in Chapter 2. In the case where two models have a relatively close performance regarding their F1 score and ROC-AUC, the model that is easier to explain is to be chosen as interpretability is key for explaining the dashboard result.

3.7 Evaluation Methodology

Models will be evaluated using the metrics listed in Table 3.4, computed on an untouched test set after stratified k-fold cross-validation on the training set. The test set will not be used for hyperparameter selection at any point in the pipeline, to avoid an optimistic bias in the reported results.

Metric	Formula	What It Measures
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall share of correct predictions

Precision	TP / (TP + FP)	Share of predicted phishing that is actually phishing
Recall	TP / (TP + FN)	Share of actual phishing correctly caught
F1-score	2 × Precision × Recall / (Precision + Recall)	Balance between precision and recall
Specificity	TN / (TN + FP)	Share of legitimate URLs correctly passed through
False Negative Rate	FN / (FN + TP)	Share of phishing URLs missed by the model
ROC-AUC	Area under the ROC curve	Ranking quality across all thresholds
PR-AUC	Area under the precision-recall curve	Ranking quality under class imbalance

Table 3.4 defines the evaluation metrics used to compare the five algorithms.

However, accuracy is not enough to solve this problem since it is possible to have either many legitimate URLs or phishing URLs in a dataset, resulting in a high rate of accuracy in the case of an algorithm that performs poorly when dealing with the minority class. False negatives, where phishing URLs are identified as legitimate, will be considered during the investigation since they are the ones that bear greater consequences as far as security is concerned.

3.8 Threshold and Risk Score Interpretation

The prediction made by the model is a probability value that lies in the range between 0 and 1. It does not imply a class label on its own. An automatic threshold of 0.5 would not be considered for the classification process. Instead, thresholds in the range between about 0.1 and 0.9 would be tried on the validation data and would be plotted against precision, recall, and F1-scores. The decision on the threshold for operation would then be made through this curve and not the test set. The tradeoff would also be noted down as follows:

Chapter 4: System Design

4.1 System Architecture

The system follows a layered architecture. A Next.js frontend sends requests to a FastAPI backend, which routes prediction requests to the machine-learning engine and stores results in the database. The ML engine applies the feature extractor, then the preprocessing pipeline, then the trained model, in that fixed order, so training and inference stay consistent by construction rather than by convention.

4.2 Data Flow Diagram

A submitted URL flows from the frontend to the backend, then to the feature extractor, which returns a fixed-length feature vector. The preprocessing pipeline applies the same scaling and encoding used during training, and the trained model returns a predicted class and probability. The backend stores the prediction and returns the result, including a short explanation, to the frontend for display.

4.3 Sequence of Operations

The following pseudocode outlines the request handling logic on the backend for a single prediction request. It is written to illustrate the order of operations rather than as final implementation code.

```
def handle_predict_request(raw_url: str) -> PredictionResult:
    validate_length(raw_url, max_len=4096)
    parsed = safe_parse(raw_url)          # no network access
    features = extract_features(parsed)   # same function used in training
    vector = preprocessing_pipeline.transform(features)
    probability = model.predict_proba(vector)
    label = 'Phishing' if probability >= threshold else 'Legitimate'
    explanation = top_contributing_features(features, model)
    save_prediction(hash(raw_url), label, probability, model_version)
    return PredictionResult(label, probability, explanation)
```

Figure 4.1: (pseudocode): sequence of operations for a single prediction request.

Every step in this sequence stays local to the machine running the backend. No step in the pipeline issues a network call to the domain named in the submitted URL, which is the central security property required by the project scope in Chapter 1.

4.4 Database Design

The database holds five core entities: users, predictions, feature_snapshots, models, and audit_logs. A prediction record stores a hashed form of the URL rather than the raw text, along with the predicted class, probability, model version, and timestamp. A feature_snapshots record stores the computed feature values for that prediction, linked by a foreign key to the prediction it belongs to, which supports later analysis without recomputing features from scratch.

4.5 API Design

The backend exposes a REST API. The primary endpoint, POST /predict, accepts a URL and returns the predicted label, probability, model version, computed features, and explanation. A sample response is shown below.

```
POST /predict
{ "url": "https://example-login-verify.com/account" }

200 OK
{
  "label": "Phishing",
  "probability": 0.87,
  "model_version": "1.0.0",
  "explanation": [
    "elevated URL entropy",
    "suspicious keyword count: 2",
    "subdomain depth above typical range"
  ]
}
```

Sample POST /predict request and response.

Supporting endpoints include GET /models, GET /predictions, GET /metrics/{model_version}, and administrative endpoints for training and model activation, which are restricted to authorized users and are never exposed to the public-facing prediction flow.

4.6 Security Design

Every submitted URL is treated as untrusted text, never as a location to visit. The backend enforces a maximum input length, validates and safely parses the URL using standard library functions only, and never issues a request to fetch it, at any point in the pipeline. Database access uses parameterized queries through an ORM to prevent injection, administrative endpoints require authentication, and all training and configuration actions are recorded in an audit log with a timestamp and the acting user.

- Input validation: reject malformed input and oversized payloads before parsing.

- No outbound fetch: the prediction code path contains no HTTP client call to the submitted URL.
- Parameterized queries: all database access goes through the ORM, never through raw string concatenation.
- Authentication: administrative endpoints require a valid session or token; prediction endpoints may be rate-limited if exposed over a network.
- Audit logging: every training run and model activation is recorded with actor, timestamp, and outcome.

Chapter 5: Implementation Plan

5.1 Project Structure

The codebase will be organized into four top-level directories: frontend for the Next.js dashboard, backend for the FastAPI service, ml for the feature extractor and training scripts, and tests for unit, integration, and security tests. Trained model artifacts and their metadata will be stored under ml/models, kept separate from source code so a retrained model can be swapped in without a code change.

```
phishing-detector/
  frontend/        Next.js dashboard (detector, result, history, model pages)
  backend/         FastAPI app, routes, request/response schemas
  ml/
    features.py    extract_features(url) - single source of truth
    preprocessing.py
    train.py
    evaluate.py
    predict.py
    models/
      phishing_model.joblib
      metadata.json
  data/            raw/ and processed/ dataset files
  tests/           test_features.py, test_api.py, test_model.py
  reports/         figures/ and metrics/ generated during evaluation
```

Planned project directory structure.

5.2 Feature Extraction Module

A single function, extract_features(url), will implement every feature listed in Table 3.2. This function will be imported directly by both the training pipeline and the prediction service, so the two never diverge. It will validate the input length, parse the URL with Python's standard library, and return a fixed-schema feature vector without making any network call. Because the function is pure, meaning its output depends only on its input and nothing else, the same URL will always produce the same feature vector, which supports the deterministic-extraction unit test described in Chapter 6.

5.3 Model Training Pipeline

The training pipeline will clean and deduplicate the dataset, apply a stratified train/test split, fit preprocessing only on the training data, run cross-validation and hyperparameter tuning for each of the five algorithms, then evaluate the selected model on the untouched test set. Every preprocessing object, such as the fitted scaler, will be saved alongside the model so the exact same transformation can be replayed at prediction time. The final model, its preprocessing

objects, and its metadata will be saved together using joblib, with the metadata file recording the algorithm name, feature schema version, threshold, training date, hyperparameters, and evaluation metrics.

5.4 Prediction Service

The FastAPI backend will load the saved model and its metadata once at startup, rather than on every request, to keep response latency low. Incoming requests are validated against a Pydantic schema before reaching the feature extractor, so a malformed request body is rejected before any processing begins. The prediction endpoint returns the response shape shown in Section 4.5, and every prediction is written to the database asynchronously so the response is not held up by the database write.

5.5 Frontend Dashboard

The Next.js frontend will provide five pages. The Detector page holds the URL input field and an analyze button, with client-side validation before the request is sent. The Result page shows the predicted label, probability, model version, and the explanation returned by the backend. The History page lists past predictions with their hashed URL, result, probability, and timestamp. The Model Information page summarizes the active model's algorithm, feature count, training date, and evaluation metrics. The Metrics page renders the confusion matrix, ROC curve, precision-recall curve, and the model comparison table produced during evaluation.

Chapter 6: Testing Plan

6.1 Testing Strategy

Testing will proceed at four levels: unit tests for the feature extractor, integration tests across the frontend, backend, and model, tests specific to the machine-learning pipeline, and security and performance tests targeting the prediction endpoint. Unit and integration tests will run through pytest and the FastAPI TestClient, and will be part of the same repository as the application code so they run on every change.

6.2 Test Case Summary

Table 6.1 lists a representative sample of the planned test cases for the feature extractor and the prediction endpoint.

Test Case	Input	Expected Outcome
Empty input	Submit an empty string as the URL	Validation error returned, no crash
Normal URL	Submit a well-formed legitimate URL	Feature vector returned with expected schema
IP-based URL	Submit http://192.168.1.1/login	has_ip_host = true
'@' in URL	Submit a URL containing '@' before the host	has_at_symbol = true
Encoded URL	Submit a URL with percent-encoded characters	num_encoded_chars > 0
URL with port	Submit a URL with an explicit port number	has_port = true
Oversized input	Submit a URL exceeding 4096 characters	Validation error, request rejected
Malformed URL	Submit unparsable text	Validation error returned, no crash
Deterministic extraction	Call extract_features twice on the same URL	Identical feature vectors returned
SQL-injection string	Submit a URL-like string containing SQL syntax	Treated as plain text; no query executed

Table 6.1 lists a sample of the planned unit and integration test cases.

6.3 Security and Performance Testing

Security tests will cover SQL-injection-style strings, cross-site-scripting-like input, oversized URLs, malformed input, unauthorized access attempts on administrative endpoints, and path traversal attempts in any file-handling code. Each of these is expected to be rejected or safely neutralized without exposing an internal error message that reveals implementation details.

Performance tests will measure feature extraction latency and end-to-end prediction latency for a single request, along with memory usage during a batch of requests. These measurements will be reported as actual figures in the final report, once the implementation is complete, rather than estimated here.

Model-level tests will check that the feature vector produced for a given URL always has the expected number of columns in the expected order, that the saved model loads without error, and that a known legitimate URL and a known phishing URL from the test set produce the expected class label. These tests catch a broken feature schema or a corrupted model file before either reaches the prediction service.

Chapter 7: Results

7.1 Phishing URL Classification

The system will provide a complete phishing URL detection workflow based exclusively on lexical and structural analysis of URL strings. A submitted URL will undergo validation and feature extraction before being processed by the trained machine-learning model. The system will classify the URL as either legitimate or phishing and provide a probability score together with a concise explanation of the factors contributing to the classification. The feature set will cover URL length, character composition, host structure, query structure, entropy, and suspicious keywords.

7.2 Machine Learning Model Evaluation

Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, and XGBoost will be evaluated using the same preprocessing, cross-validation, hyperparameter-tuning, and held-out testing methodology. The comparison will include accuracy, precision, recall, F1-score, specificity, false-negative rate, ROC-AUC, and PR-AUC. The evaluation will provide a measurable comparison of the five algorithms, with particular attention to false negatives because of their security implications.

7.3 Feature and Threshold Analysis

The feature ablation study will demonstrate the contribution of the six URL feature groups: length, character count, host structure, query structure, entropy, and suspicious keywords. Feature groups will be progressively removed and the resulting changes in classification performance will be measured. In addition, multiple classification thresholds will be evaluated using validation data to compare precision, recall, and F1-score. This will provide a basis for understanding both feature contribution and the trade-off between detecting phishing URLs and incorrectly flagging legitimate URLs.

7.4 Application and Dashboard

The completed application will integrate URL validation, feature extraction, machine-learning inference, result storage, and visualization into a single workflow. The FastAPI backend will provide the prediction service, while the Next.js dashboard will display the predicted class, probability score, model version, and feature-based explanation. The dashboard will also

provide prediction history, model information, confusion matrices, ROC curves, precision-recall curves, and model comparison results.

7.5 Security, Testing and Performance

The prediction pipeline will operate without fetching or opening the submitted URL and will not depend on external services. The same feature-extraction function will be used during training and inference to maintain consistency. Testing will cover feature extraction, model loading, prediction behaviour, malformed and oversized inputs, injection-style inputs, unauthorized administrative access, and other security conditions. Performance testing will measure feature-extraction latency, end-to-end prediction latency, and memory usage, with the final report presenting the measurements obtained from the completed implementation.

Chapter 8: Conclusion and Future Enhancement

8.1 Conclusion

This synopsis outlines a phishing detection system that classifies URLs using lexical and structural features alone, without any dependency on external APIs or live page access. The design keeps the training and inference pipelines consistent by sharing one feature-extraction function, evaluates five algorithms under a documented cross-validation protocol, and defers every performance claim to measured results from the completed experiments rather than to figures borrowed from other papers.

The planned system is scoped to be achievable within an MCA final-year project timeline, with a clear separation between what is implemented now, described through Chapters 1 to 6, and what is proposed as an extension, described in Section 7.2. This separation is intended to keep the academic claims made in the final report conservative and traceable to actual, reproducible work.

8.2 Future Work

- Character-level deep-learning models, such as a CNN or a URLNet-style architecture, trained directly on raw URL strings instead of hand-engineered features.
- Combining URL features with HTML or DNS-based signals, once the project scope is extended to allow controlled page access in a sandboxed environment.
- A browser extension that applies the trained model in real time as a user navigates, warning before a page finishes loading.
- Monitoring for model drift as new phishing URL patterns emerge after deployment, with periodic retraining on freshly labeled data.
- Adversarial testing, to measure how easily the model can be fooled by small, deliberate changes to a phishing URL's structure.

References

[1] UCI Machine Learning Repository, “Phishing Websites Dataset,” University of California, Irvine.

[2] PhiUSIIL Phishing URL Dataset, published dataset for phishing URL classification research.

[3] Pedregosa, F. et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, 2011.

[4] Chen, T. and Guestrin, C., “XGBoost: A Scalable Tree Boosting System,” Proc. 22nd ACM SIGKDD, 2016.

[5] Le, H. et al., “URLNet: Learning a URL Representation with Deep Learning for Malicious URL Detection,” arXiv preprint, 2018.

[6] FastAPI documentation, https://fastapi.tiangolo.com/

[7] Next.js documentation, https://nextjs.org/docs

Annexure

A. Complete URL Feature Schema

The complete feature schema is documented in Table 3.2 of Chapter 3. It lists all 30 planned features, grouped into length, character-count, host-structure, query-structure, entropy, and suspicious-keyword categories, each with its data type and the reasoning behind its inclusion.

B. Development Schedule

Table Annexure-B outlines the planned ten-week schedule for implementation, evaluation, and documentation.

Week	Planned Activity
1	Problem definition, literature survey, dataset selection
2	Dataset cleaning and preprocessing
3	URL feature extractor implementation
4	Feature extractor unit tests
5	Baseline models: Logistic Regression and Decision Tree
6	Random Forest, SVM, and XGBoost
7	Cross-validation, feature selection, and ablation study
8	Backend prediction service and database
9	Frontend dashboard and explainability display
10	Testing, security review, documentation, and viva preparation

Table Annexure-B: ten-week development schedule.